

HLDD AI Agent

Application Architecture Documentation

Generated: May 2026

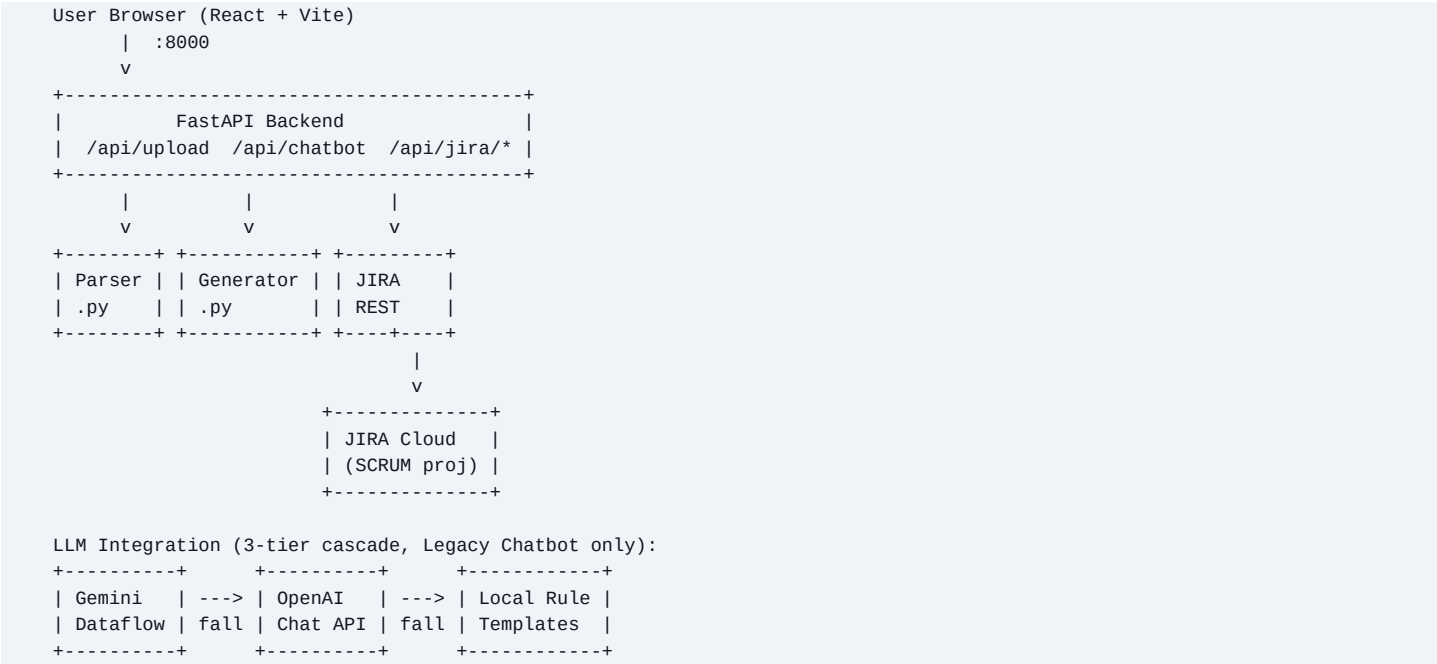
Repository: github.com/jaiminbprajapatireminder-eng/SDLC-HLDD-AI

1. High-Level Architecture Overview

The HLDD AI Agent is a full-stack web application that ingests High-Level Design Documents, extracts structured project data, generates a delivery plan (features, stories, test cases, architecture diagrams), and provides an interactive dashboard with LLM-powered chatbot and JIRA synchronization.

Stack: React + Vite (frontend) | FastAPI (backend) | Gemini/OpenAI (LLM) | JIRA Cloud (project mgmt)
No database is used -- state is held in an in-memory dictionary on the backend and in React state on the frontend.

Component Diagram



2. Functionality Implemented

HLDD Upload & Parsing (parser.py)

- Supports .txt, .md, .docx, .pdf uploads. Extracts: title, summary, functional requirements, technology stack, acceptance criteria, architecture notes, project management tool, repository, CI/CD pipeline. Uses regex + keyword matching (NO LLM).
- Takes parsed data and generates: features (max 4 from requirements), development stories (3-4 per feature based on tech stack), testing stories (1 per dev story), Mermaid architecture diagram (nodes + edges), recommended project file structure. Uses rule-based logic (NO LLM).
- 7 tabs: Dashboard (summary stats), Features (table with CRUD), Stories (table with CRUD), Testing (table with CRUD), Architecture (Mermaid.js interactive diagram), Structure (file tree), Chatbot (LLM QA). Export to PDF, Excel, TXT.
- 3-tier pipeline for classic QA: (1) Gemini 3-stage dataflow (Analyze -> Reason -> Generate), (2) OpenAI single-call fallback (GPT-4.1-mini / GPT-4o-mini), (3) Local rule-based templates. Context is built from the parsed HLDD payload. Single-shot (no multi-turn agent loop).
- 3 endpoints: create-features (Epics), create-stories (Stories linked to Epics), create-testing-stories (Subtasks linked to Stories). JIRA Cloud REST API v3.
- Add, edit, delete features/stories/testing stories from the UI. Changes are local to the React state and not persisted on the backend.

3. LLM Usage -- Chatbot Detailed Breakdown

The legacy chatbot endpoint (/api/chatbot) implements a 3-tier cascade with automatic LLM selection based on the project's technology stack.

Tier 1: Gemini Dataflow (gemini_dataflow.py)

Model Pipeline: gemini-2.0-flash -> gemini-2.0-flash -> gemini-2.5-flash

```
Stage 1 - ANALYZE (gemini-2.0-flash, temp=0.1, max_tokens=1024)
  Prompt: classify question topic, scope, key terms, intent
  Output format: TOPIC / SCOPE / KEY_TERMS / INTENT

Stage 2 - REASON (gemini-2.0-flash, temp=0.2, max_tokens=2048)
  Prompt: step-by-step reasoning with 5 structured questions
  Output: 1. What is being asked? 2. Relevant context?
          3. Options? 4. Tradeoffs? 5. Best recommendation?

Stage 3 - GENERATE (gemini-2.5-flash, temp=0.15, max_tokens=4096)
  Prompt: produce structured final response
  Output sections: Summary / Details / Recommendation / Next Steps

Fallback: If any stage fails, a single call to gemini-2.5-flash is made.
API: Uses direct REST (urllib), NOT the google-generativeai SDK.
```

Tier 2: OpenAI Fallback (main.py - call_openai)

Model: gpt-4.1-mini or gpt-4o-mini (selected by recommend_llm() based on tech stack)

Single-shot call with system instruction + user message containing context.

System instruction covers: Recommendation, Setup, Alternatives, Efficiency, Security, Turnaround time, Cost breakdown.

Tier 3: Local Rule Fallback (main.py - build_local_response)

NO LLM call. Pure Python template engine (~40KB of dicts/if-else).

Question type detection via keyword matching:

- backend_alternative, backend_mobile, frontend_alternative
- cloud_alternative, database_alternative, cost, general

Includes full cloud pricing tables (AWS/Azure/GCP) hardcoded in the source.

Chatbot LLM Recommendation Logic

```
Chatbot (/api/chatbot) recommendation:
  Tech stack contains 'Google Cloud' or 'GCP'    -> Gemini Dataflow
  Tech stack has React + FastAPI + PostgreSQL + Docker -> GPT-4.1-mini
  Tech stack has AWS/Azure/GCP                    -> GPT-4o-mini
  Default                                          -> GPT-4o-mini
```

Out-of-Scope Detection

Before any LLM call, the prompt passes through is_out_of_scope() which checks if the question is related to the HLDD project context. Irrelevant prompts (e.g., 'write a poem') return an 'Out of Scope' response without consuming API credits.

Context Stuffing (build_chat_context())

The chatbot does NOT use RAG. Instead, the entire parsed HLDD payload is serialized to a plain-text string and prepended to every LLM prompt. This works for small documents but will break on large HLDD files that exceed the model's context window.

```
Context includes: Title, Summary, Technology stack,
Project management tool, Repository, CI/CD pipeline,
Functional requirements, Acceptance criteria,
Architecture notes, Project structure,
Feature count, Story count, Testing story count
```

4. Full Technology Stack

Backend

- Python 3.9+ — FastAPI framework for all REST API endpoints
- Google Gemini API — 3-stage dataflow for chatbot (Analyze -> Reason -> Generate)
- OpenAI API — GPT-4.1-mini / GPT-4o-mini fallback for chatbot
- JIRA Cloud REST API v3 — Epics (10001), Stories (10004), Subtasks (10002)
- PyPDF2 / python-docx — Multi-format file parsing (.pdf, .docx)
- python-dotenv — Auto-load .env file with API keys
- fpdf2 — Offline PDF generation (this document)

Frontend

- React 18 — Hooks-based UI (useState, useEffect, useCallback, useMemo, useRef)
- Vite — Build tool and dev server with hot-reload
- Mermaid.js — Architecture diagram rendering from JSON/MMC definition
- jsPDF + html2canvas — PDF export from DOM elements
- xlsx — Excel (.xlsx) export for Features, Stories, Testing tabs

Infrastructure / DevOps

- Git + GitHub — Version control and CI/CD with Copilot Agent
- JIRA Cloud (SCRUM project) — Issue tracking with Epics, Stories, Subtasks
- Bitbucket — Repository hosting (parsed from HLDD)

5. Data Flow Diagrams

5.1 Upload -> Dashboard Flow

```

User uploads .txt/.docx/.pdf
|
v
POST /api/upload
|
v
_read_upload() -- extracts text from file
|
v
parse_hldd() -- regex parsing, NO LLM
|
v
generate_project_plan() -- rule-based gen, NO LLM
|
v
LATEST_PAYLOAD (in-memory dict)
|
v
Response JSON -> React renders 7-tab dashboard
Dashboard/Features/Stories/Tests/Architecture/Structure/Chatbot

```

5.2 Chatbot Request Flow

```

User types question in chatbot tab
|
v
POST /api/chatbot { prompt, priority }
|
v
is_out_of_scope(prompt) -----> True --> return 'Out of Scope'
| False
|
v
recommend_llm(payload)
|
v
call_gemini() (if Gemini recommended)
| Success? -> return response
| Fail? -> fall through
|
v
call_openai() (if OpenAI recommended)
| Success? -> return response
| Fail? -> fall through
|
v
build_local_response() -- hardcoded templates
| Always works
|
v
Return { response, source, recommended_llm }

```

5.3 JIRA Sync Flow

```

User clicks 'Create in JIRA' (per tab)
|
v
Features tab -> POST /api/jira/create-features
| Creates Epics in JIRA (SCRUM project)
| Returns mapping { feature_id -> SCRUM-XX }
| Stored in frontend: jiraEpicMapping
|
v
Stories tab -> POST /api/jira/create-stories
| Sends jiraEpicMapping in request body
| Creates Stories with parent=Epic key
| Returns mapping { story_id -> SCRUM-YY }
| Stored in frontend: jiraStoryMapping
|
v
Testing tab -> POST /api/jira/create-testing-stories
| Sends jiraStoryMapping in request body
| Creates Subtasks with parent=Story key
| Returns results

```

6. Project File Structure

```
HLDD-AI/
+-- backend/
|   +-- app/
|   |   +-- main.py           -- FastAPI app, all endpoints
|   |   +-- parser.py        -- HLDD document parsing (regex)
|   |   +-- generator.py     -- project plan generation (rules)
|   |   +-- gemini_dataflow.py -- 3-stage Gemini pipeline
|   |   +-- models.py        -- Pydantic models
|   |   +-- __init__.py
|   +-- tests/
|   |   +-- test_hldd_pipeline.py
|   |   +-- test_chatbot_response.py
|   +-- requirements.txt
|   +-- .gitignore
+-- frontend/
|   +-- src/
|   |   +-- App.jsx           -- Main React component (all tabs)
|   |   +-- main.jsx         -- Entry point
|   |   +-- index.css        -- All styles
|   |   +-- download.js      -- PDF/Excel/TXT export
|   +-- public/
|   |   +-- sample-hldd-template.txt
|   +-- index.html
|   +-- package.json
|   +-- vite.config.js
|   +-- .gitignore
+-- .env                     -- API keys (gitignored)
+-- .gitignore
+-- README.md
+-- prompt.txt
+-- generate_arch_pdf.py     -- This document generator
+-- HLDD_AI_Architecture.pdf -- Generated arch doc
```

7. Environment Variables

Variable	Required	Description

GEMINI_API_KEY	Preferred	Gemini key for chatbot 3-stage dataflow
JIRA_EMAIL	For JIRA	JIRA Cloud email address
JIRA_API_TOKEN	For JIRA	JIRA Cloud API token
OPENAI_API_KEY	No	OpenAI key (chatbot fallback)

8. Current Limitations

- No true RAG -- context stuffing limits HLDD document size
- No agentic loop -- chatbot is single-shot QA, no multi-turn reasoning
- JIRA sync is manual (3 separate UI buttons), no autonomous workflow
- No database -- LATEST_PAYLOAD is in-memory, lost on backend restart
- Local fallback is ~40KB of hardcoded Python templates -- brittle
- No auth/authentication -- open endpoints, CORS allows all origins
- No async Gemini calls -- synchronous urllib may block the event loop
- No streaming -- chatbot response returns as a single block

Direct REST over SDK for chatbot

- Gemini and OpenAI are called via raw urllib (not google-generativeai or openai Python SDK). This avoids SDK dependency issues and keeps control over HTTP behavior.

- Since the JIRA project is team-managed (simplified=true), the standard Epic Link custom field (customfield_10014) is unavailable. Links are created via the 'parent' field in the issue creation payload instead.

- JIRA sync mappings (feature->Epic key, story->Story key) are stored in React state, not on the backend. This avoids needing a database but means mappings are lost on page refresh.

- Gemini (dataflow) -> OpenAI (fallback) -> Local rules (last resort). This ensures the app always returns a response even if all API keys are unset.

- Since HLDD documents are small (typically <50KB), context stuffing is simpler to implement and has lower latency than a full RAG pipeline with vector embeddings.

- All stories are capped at 3 story points regardless of complexity. Point estimation uses word count heuristics (<=14 words = 1, <=28 words = 2, else 3).